

Lab 5 - Мікросервіси з використанням Service Discovery та Config Server

Github: https://github.com/Sirska-Mariia/homework1/tree/micro_consul

Демонстрація створення ресурсів за допомогою маніфестів (`kubectl apply -f`).

```
PS C:\апз\homework1\banking_system_> kubectl apply -f k8s/
configmap/banking-config created
service/counter-service created
deployment.apps/counter-service created
service/facade-service created
deployment.apps/facade-service created
service/hazelcast-service created
deployment.apps/hazelcast created
service/logging-service created
deployment.apps/logging-service created
service/postgres-db created
deployment.apps/postgres-db created
```

На скріншоті вивід команди `kubectl get pod` підтверджує, що всі необхідні Pod-и (counter-service, facade-service, hazelcast, logging-service, postgres-db) запущені та мають статус Running.

```
PS C:\апз\homework1\banking_system_> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
counter-service-c5c6d87c5-q5n2m    1/1     Running   0           33m
facade-service-64fc95dbf7-tfccd     1/1     Running   0           22s
hazelcast-67f6699d4f-wddhq         1/1     Running   0           110m
logging-service-785cc4489b-dkndh    1/1     Running   0           25m
logging-service-785cc4489b-r697c    1/1     Running   0           32m
logging-service-785cc4489b-snb66    1/1     Running   0           33m
postgres-db-58f5f45778-69kqq       1/1     Running   0           110m
PS C:\апз\homework1\banking_system_>
```

Демонстрація роботи Service Discovery (Endpoints):

```
deployment.apps/logging-service restarted
PS C:\апз\homework1\banking_system_> kubectl get endpoints
NAME                               ENDPOINTS                                AGE
counter-service                    10.1.0.20:8000                           78m
facade-service                      10.1.0.21:8000                           78m
hazelcast-service                  10.1.0.13:5701                           78m
kubernetes                          192.168.65.3:6443                        8h
logging-service                    10.1.0.22:8000,10.1.0.23:8000,10.1.0.24:8000 78m
postgres-db                        10.1.0.12:5432                           78m
PS C:\апз\homework1\banking_system_> 
```

Вивід команди `kubectl get endpoints` підтверджує автоматичну реєстрацію мікросервісів у мережі кластера. На скріншоті видно, що сервіс `logging-service` зареєстрував три різні IP-адреси, що відповідає трьом працюючим екземплярам логера.

Збереження налаштувань у Key/Value (ConfigMap):

```
PS C:\апз\homework1\banking_system_> kubectl describe configmap banking-config
Name:          banking-config
Namespace:     default
Labels:        <none>
Annotations:   <none>

Data
====
DB_URL:
----
postgresql://postgres:password@postgres-db:5432/bank_db
POSTGRES_DB:
----
bank_db
POSTGRES_USER:
----
postgres
COUNTER_SERVICE_URL:
----
http://counter-service:8000
HZ_ADDRESSES:
----
hazelcast-service:5701
HZ_CLUSTER_NAME:
----
dev
LOGGING_SERVICE_URL:
----
http://logging-service:8000
MQ_QUEUE_NAME:
----
transaction-queue
POSTGRES_PASSWORD:
----
password

BinaryData
====

Events:  <none>
PS C:\апз\homework1\banking_system_> █
```

Скріншот доводить, що всі параметри систем зберігаються централізовано як словник Key/Value та передаються у мікросервіси середовищем Kubernetes.

Обробка вхідної транзакції (POST-запит):

POST

/transaction

Create Transaction

Parameters

No parameters

Request body required

Edit Value | Schema

```
{ "user_id": "Mariia", "amount": 1000 }
```

Execute

Responses

localhost:30080/docs#/default/create_transaction_transaction_post

ExecuteClear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:30080/transaction' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{ "user_id": "Mariia", "amount": 1000 }'
```

Request URL

http://localhost:30080/transaction

Server response

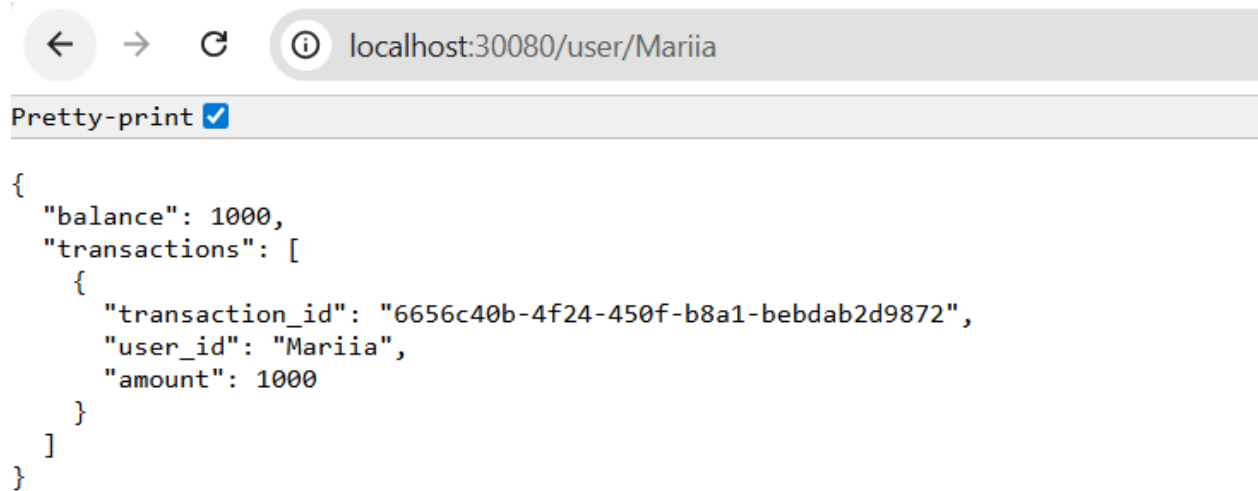
Code	Details
200	Response body <pre>{ "status": "Transaction accepted (queued)", "transaction_id": "6656c40b-4f24-450f-b8a1-bebdsb2d9872" }</pre> Download Response headers <pre>content-length: 98 content-type: application/json date: Wed, 13 May 2026 20:07:27 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Зроблено POST-запит на ендпоінт /transaction для користувача "Mariia" на суму 1000. Запит відправлено через відкритий порт NodePort (localhost:30080) та отримано успішну відповідь зі статусом 200.

Перевірка синхронізації даних (GET-запит):



Результат GET-запиту до `localhost:30080/user/Mariia` у браузері. Відповідь підтверджує, що транзакція успішно пройшла, логер зберіг дані транзакції, а лічильник вичитав повідомлення з черги та коректно оновив баланс користувача до 1000.

Продемонструвати, що у випадку відключення екземпляру певного мікросервісу, це буде відображатися у Consul/K8s (відключений екземпляр сервісу змінить статус), а виклики будуть перенаправлятися до інших працюючих екземплярів.

Тест на відмовостійкість та самовідновлення:

```

PS C:\анз\homework1\banking_system_> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
counter-service-c5c6d87c5-q5n2m    1/1     Running   0           6m57s
facade-service-65f5f58697-np4sn    1/1     Running   0           6m57s
hazelcast-67f6699d4f-wddhq        1/1     Running   0           84m
logging-service-785cc4489b-gggqf    1/1     Running   0           6m33s
logging-service-785cc4489b-r697c    1/1     Running   0           6m47s
logging-service-785cc4489b-snb66    1/1     Running   0           6m57s
postgres-db-58f5f45778-69kqq       1/1     Running   0           84m
PS C:\анз\homework1\banking_system_> kubectl delete pod logging-service-785cc4489b-gggqf
pod "logging-service-785cc4489b-gggqf" deleted
PS C:\анз\homework1\banking_system_> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
counter-service-c5c6d87c5-q5n2m    1/1     Running   0           8m8s
facade-service-65f5f58697-np4sn    1/1     Running   0           8m8s
hazelcast-67f6699d4f-wddhq        1/1     Running   0           85m
logging-service-785cc4489b-dkndh    1/1     Running   0           11s
logging-service-785cc4489b-r697c    1/1     Running   0           7m58s
logging-service-785cc4489b-snb66    1/1     Running   0           8m8s
postgres-db-58f5f45778-69kqq       1/1     Running   0           85m
PS C:\анз\homework1\banking_system_> 

```

Демонстрація виконання вимоги щодо відключення екземпляру. Виконано видалення одного з Pod-ів логера командою `kubectl delete pod logging-service-785cc4489b-gggqf`. Повторний виклик `kubectl get pods` підтверджує, що Kubernetes миттєво зреагував на втрату та створив новий Pod (`logging-service-785cc4489b-dkndh`).

Тестування продуктивності

```
PS C:\апз\homework1\banking_system_ > & C:/Users/olsir/AppData/Local/Programs/Python/Python311/python.exe c:/апз/homework1/banking_system_/test.py
```

```
Запуск: Сценарій 1 (Різні рахунки)
Налаштування: 10 клієнтів по 10000 запитів
Результати Сценарій 1 (Різні рахунки):
  Загальний час: 1177.88 сек
  RPS (запитів/сек): 84.90
  Сер. час виклику Logging: 0.0001 сек
  Сер. час виклику Counter: 0.0015 сек
```

```
Запуск: Сценарій 2 (Один рахунок)
Налаштування: 10 клієнтів по 10000 запитів
Результати Сценарій 2 (Один рахунок):
  Загальний час: 3271.06 сек
  RPS (запитів/сек): 30.57
  Сер. час виклику Logging: 0.0001 сек
  Сер. час виклику Counter: 0.0021 сек
PS C:\апз\homework1\banking_system_ > █
```

Порівняння з минулими лабораторними:

Test scenarios	Task 1	Task 3	Task 5
10 accounts	Total time: 1797.23 сек	Total time: 2921.88 сек	Total time: 1177.88 сек
	logging-service contribution: 0.1236 сек	logging-service contribution: 0.2422 сек	logging-service contribution: 0.0001 сек

	counter-service contribution: 0.1236 сек	counter-service contribution: 0.2422 сек	counter-service contribution: 0.0015 сек
1 account	Total time: 2061.00 сек	Total time: 3329.85 сек	Total time: 3271.06 сек
	logging-service contribution: 0.1148 сек	logging-service contribution: 0.2211 сек	logging-service contribution: 0.0001 сек
	counter-service contribution: 0.1148 сек	counter-service contribution: 0.2211 сек	counter-service contribution: 0.0021 сек

У Завданні 5 час опрацювання запитів мікросервісами `logging-service` та `counter-service` зменшився майже до нуля (0.0001 сек та 0.0015 сек відповідно). Це пояснюється впровадженням асинхронного виконання та використанням `Message Queue`. Фасад більше не чекає на збереження даних у базу даних і миттєво повертає клієнту відповідь зі статусом `queued`.

Загальний час виконання (10 accounts): використання балансування навантаження `Kubernetes` між трьома екземплярами `logging-service` дозволило суттєво зменшити загальний час виконання для 10 різних рахунків (1177.88 сек) порівняно з монолітною БД у Завданні 3.

Особливості сценарію "1 account": для одного рахунку загальний час залишається високим через особливості блокування при оновленні одного й того ж запису в базі даних та черзі, однак клієнт при цьому не блокується і продовжує безперебійно відправляти запити.